AULA 08
FUNÇÕES E MÓDULOS

Programação em Python • Parte 2 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Funções e Módulos • 8 horas
Professor: Msc. Hygo Sousa de Oliveira

Roteiro da aula

O que vamos ver nesta aula

1. Revisão: o que o código imprime?
2. Módulos: importar e criar o seu
3. Pacotes: organizar em pastas
4. Bibliotecas padrão e JSON
5. Exemplo guiado: validação de entradas
6. Atividades da aula

Revisão: o que o código imprime?

Preveja a saída antes de executar

QUESTÃO 1

```
def f(x, lista=[]):
    lista.append(x)
    return lista

print(f(1))
print(f(2))
```

QUESTÃO 2

```
x = 5

def h():
    x = 10
    return x

print(h(), x)
```

EM SALA Peça que a turma escreva a previsão no caderno, compare com os colegas e só então execute.

Módulos: importação

Reaproveitar código que já existe, seu ou da biblioteca padrão

Forma	Uso
`import math`	`math.sqrt(16)`
`from math import sqrt`	`sqrt(16)`
`import statistics as st`	`st.mean([4, 8, 9])`

importacao.py

```
import math
from math import sqrt
import statistics as st

print(math.ceil(4.2), sqrt(81), st.mean([4, 8, 9]))
```

SAÍDA

```
5 9.0 7
```

ATENÇÃO Evite `from modulo import *`: despeja nomes no seu código e causa conflitos.

Criando e usando seu módulo

Todo arquivo .py é um módulo

utilidades.py

```python
PI = 3.14159

def area_circulo(raio):
    return PI * raio ** 2

def celsius_para_f(c):
    return c * 9 / 5 + 32
```

principal.py

```python
import utilidades

print(utilidades.area_circulo(2))
print(utilidades.celsius_para_f(30))
```

SAÍDA

```
12.56636
86.0
```

NOTA O módulo precisa estar na mesma pasta (ou em um pacote). O nome do arquivo, sem `.py`, é o nome do import.

Módulo ou programa? O __name__

O bloco if __name__ == "__main__" roda só na execução direta

python utilidades.py

```python
def area_circulo(raio):
    return 3.14159 * raio ** 2

print(__name__)
if __name__ == "__main__":
    print(area_circulo(1))
```

SAÍDA

```
__main__
3.14159
```

python principal.py

```python
import utilidades

print(utilidades.area_circulo(2))
```

SAÍDA

```
utilidades
12.56636
```

DICA Ao importar, `__name__` vale o nome do módulo e o teste não roda. Use o bloco para testar o módulo sozinho.

Pacotes: organizando em pastas

Um pacote é uma pasta com módulos de responsabilidade única

estrutura

```
projeto/
├── main.py
└── loja/
    ├── __init__.py
    ├── precos.py
    └── estoque.py
```

NOTA `__init__.py` marca a pasta como pacote e pode ficar vazio. O projeto Sistema de Notas (N2) usa a mesma ideia com o pacote `funcoes/`.

Usando um pacote

from pacote.modulo import nome

loja/precos.py

```python
def com_desconto(preco, pct):
    return preco * (1 - pct / 100)
```

loja/estoque.py

```python
def tem_estoque(qtd, pedido):
    return qtd >= pedido
```

main.py

```python
from loja.precos import com_desconto
from loja import estoque

print(com_desconto(200, 15))
print(estoque.tem_estoque(10, 12))
```

SAÍDA

```
170.0
False
```

Bibliotecas padrão

Já vêm com o Python: sem instalar nada

math
Raiz, arredondamentos e constantes como pi.

random
Números e escolhas aleatórias.

datetime
Datas, horas e diferenças entre elas.

os
Arquivos, pastas e sistema operacional.

json
Ler e gravar dados em formato JSON.

collections
Counter, defaultdict e outras coleções.

Bibliotecas padrão em ação

random com seed, datetime e collections

padrao.py

```
import random
from datetime import date
from collections import Counter

random.seed(7)
print(random.randint(1, 6), random.choice(["Ana", "Bia", "Caio"]))
print((date(2026, 12, 25) - date(2026, 10, 19)).days)
print(Counter("banana"))
```

SAÍDA

```
3 Ana
67
Counter({'a': 3, 'n': 2, 'b': 1})
```

DICA `random.seed()` fixa a sequência: útil para reproduzir resultados em sala e em testes.

JSON: salvar e carregar dados

Formato de texto para guardar listas e dicionários em arquivo

json_basico.py

```
import json

dados = {"nome": "caneta", "preco": 2.5}

with open("dados.json", "w", encoding="utf-8") as f:
    json.dump(dados, f, ensure_ascii=False, indent=2)
with open("dados.json", encoding="utf-8") as f:
    print(json.load(f)["nome"])
print(json.dumps(dados))
```

SAÍDA

```
caneta
{"nome": "caneta", "preco": 2.5}
```

NOTA `dump`/`load` usam arquivos; `dumps`/`loads` usam texto (string).

Exemplo guiado: validação de entradas

Função de validação + dict comprehension

DESAFIO Descarte preços inválidos: texto que não é número ou valor menor ou igual a zero.

validar.py

```python
def validar_preco(texto):
    try:
        preco = float(texto)
    except ValueError:
        return None
    return preco if preco > 0 else None

entradas = {"caneta": "2.5", "lápis": "abc", "caderno": "18"}
validos = {n: validar_preco(t) for n, t in entradas.items()}
print({n: p for n, p in validos.items() if p is not None})
```

SAÍDA

```
{'caneta': 2.5, 'caderno': 18.0}
```

Atividades da Aula 8

Módulo 4: conclusão do Registry (N1), Sistema de Notas (N2) e checkpoint

- **Projeto Registry Pattern (N1) – Atividade 4**
  Motor de descontos de e-commerce
- **Projeto Registry Pattern (N1) – Atividade 5**
  Processamento de eventos com Registry completo e troca de handler em runtime
- **Projeto Sistema de Lançamento de Notas (N2)**
  Validações de entrada, persistência em JSON (funcoes/arquivo.py) e orquestração em main.py
- **Checkpoint de Funções (7 questões)**
  Parte do checkpoint dos Módulos 3 e 4, entregue via GitHub

GIT Enunciados, instruções de execução e de entrega:
github.com/Hygo/python-fullstack-iteam-2sem

Obrigado(a)!

Próxima aula: Programação Orientada a Objetos – classes e objetos
ITEAM – Instituto Tecnológico Educacional da Amazônia